

Linux USB gadget mode as a process boundary

ConfigFS, FunctionFS, the UDC, descriptor-driven endpoint discovery, file-descriptor capabilities, and disposable device workers.

- 01 The supervisor constructs and binds the device.
- 02 The worker receives open USB resources, never privileged paths.
- 03 The kernel carries payloads directly between host and worker.
- 04 Worker exit is the complete reconnect and reset primitive.

Offline companion to [docs/usb-gadget-architecture.md](#)

01 / MENTAL MODEL

The four layers

Layer	What it is	Responsibility
UDC	USB Device Controller hardware + Linux driver	Moves electrical USB traffic and exposes one bindable controller name.
ConfigFS	Kernel configuration filesystem	Builds the host-visible gadget: identity, configurations, function links, and UDC binding.
FunctionFS	Kernel API exposed as files	Accepts interface descriptors on ep0, creates endpoint files, and reports runtime events.
Worker	Unprivileged device implementation	Handles CTAP, CCID, Trezor messages, crypto, policy, display, buttons, and state.

The resulting stack

```
host application / OS class driver
| USB packets
v
physical link -> UDC -> composite gadget
|
FunctionFS endpoint FDs
|
v
unprivileged worker
```

ConfigFS describes and assembles the device. FunctionFS supplies user-space interfaces. The UDC makes the assembled gadget electrically present to the host.

Binding is the last privileged action

The gadget remains invisible while it is being constructed. Writing the selected UDC name to the gadget's ConfigFS UDC attribute is the logical equivalent of plugging in the cable. Writing an empty value disconnects it.

Descriptors become capabilities

The worker constructs a typed USB personality and sends it as CBOR in a Configure record. The root-owned profile describes launch arguments and approved local resources. The supervisor validates the personality, creates ConfigFS and FunctionFS, publishes descriptors, and opens the resulting endpoint files.

Declaration	Supervisor responsibility	Worker receives
Device identity	Publish VID/PID, strings and configuration	Its accepted generation
Interface and endpoint map	Validate directions, types and packet sizes	Typed endpoint metadata and FDs
FunctionFS descriptors	Write descriptor and string blobs to ep0	Only data endpoints
Local named resources	Open approved SPI/I2C devices and exact GPIO line groups	Narrow resource handles

The supervisor retains ep0

FunctionFS ep0 remains the privileged control plane. The supervisor reads bind, enable, disable, unbind, suspend and resume events and forwards typed UsbBusEvent records. Setup requests arrive at the worker as UsbControlRequest records; the worker sends UsbControlResponse records without owning ep0.

The complete USB declaration exists before binding. Runtime control replies are protocol responses, not late descriptor publication. Both YubiKey FIDO HID and CCID use one FunctionFS composite function.

The worker defines the device; the supervisor validates and publishes it. Application payloads then travel directly through data endpoint FDs.

A typed resource protocol

An AF_UNIX/SOCK_SEQPACKET socket is installed on fixed worker FD 3. UGSP version 1 records have a 20-byte header plus a bounded body. SCM_RIGHTS carries the declared count of attached descriptors. Generation and request identifiers prevent stale control replies from crossing incarnations.

```
0..3 magic: UGSP
4 version: 1
5 message kind
6..7 attached FD count
8..11 generation
12..15 request identifier
16..19 body length
20.. body (CBOR for declarations)
```

Direction	Record	Purpose
Supervisor -> worker	InitialResources	Named, approved local resources
Worker -> supervisor	Configure	Typed USB personality
Supervisor -> worker	UsbEndpoints	Endpoint metadata and data FDs
Worker -> supervisor	Serving	Ready before UDC binding
Supervisor -> worker	UsbBusEvent / UsbControlRequest	Lifecycle and setup
Worker -> supervisor	UsbControlResponse	Setup reply
Both directions	Quiesce / Quiesced	Stop endpoint work and flush

A transferred FD references the same open file description, including access mode and shared offsets. It grants only the operations supported by that handle. GPIO resources contain exact claimed line groups; the worker never receives a broad GPIO-chip FD.

The cleared environment carries state/runtime directory paths, not resource descriptor numbers. Handles arrive through the control socket.

One worker, one incarnation

```
START WORKER with reduced credentials
| InitialResources
v
AWAIT Configure + UsbPersonality
| validate; create ConfigFS / FunctionFS
v
SEND UsbEndpoints + data FDs
|
v
AWAIT Serving -> bind UDC -> HOST ENUMERATION
|
+--> forward lifecycle / setup on control socket
| worker exit / EOF
v
CLEAN: unbind, close, reap, unmount, remove
+--> fresh worker incarnation
service stop: clean -> supervisor exits
```

Serving means the worker has validated the endpoint map and initialized its protocol state. Only then does the supervisor bind the UDC. Virtual YubiKey receives five FunctionFS data endpoints: two FIDO interrupt endpoints and three CCID endpoints. There is no separate post-bind ConfigFS HID handoff.

The supervisor retains ep0 and forwards lifecycle and setup records. The worker handles ordinary protocol traffic directly on its endpoint handles. Quiesce stops endpoint work and flushes durable state before a controlled teardown.

A firmware reconnect is worker exit. The supervisor unbinds first, destroys the incarnation, and launches a fresh Unix process. A service stop performs final cleanup and exits.

Process creation is the reset primitive for volatile threads, buffers and capabilities. Persistent applet records survive ordinary restart.

The supervisor leaves the data plane

After startup, the kernel moves USB payloads directly between the host controller and worker-held FDs. The privileged process handles metadata and lifecycle only; it never sees CTAP frames, APDUs, Trezor protobuf messages, PINs, seeds, keys, or YubiHSM commands.

Device	Kernel surface	Worker traffic
Virtual YubiKey	FunctionFS HID + CCID	FIDO reports and CCID data endpoints; setup over the control socket
Virtual Trezor	FunctionFS vendor interface	64-byte interrupt OUT/IN packets; forwarded setup and lifecycle
Virtual YubiHSM	FunctionFS vendor bulk interface	Bulk request/response frames; forwarded setup and lifecycle

Virtual YubiKey

The shared core implements Management, FIDO2, PIV, OpenPGP, YubiHSM Auth, and Issuer Security Domain. GlobalPlatform SCP03/SCP11a/b/c protects every selectable CCID applet. HID and CCID FIDO routes share one authenticator; embedded readers use the same core and per-applet storage runtime.

The worker advertises the compatibility manufacturer Yubico and product YubiKey Gadget FIDO+CCID for controlled local testing. Gadget distinguishes the virtual reader. Host tools include browsers, ykman, yubico-piv-tool, GnuPG and pkcs11rs. This identity does not imply vendor endorsement.

Virtual Trezor

The worker owns its USB personality and direct OUT/IN endpoint traffic. Named display and button resources are restricted to the profile's declared device and GPIO line groups. Upstream firmware composes the 128x64 framebuffer; button handling uses edge events and atomic snapshots. Orderly exit clears the display.

Virtual YubiHSM

The same FunctionFS pattern supports a vendor bulk device. Its worker supplies the USB personality and owns commands, sessions, objects, capabilities, audit behavior, and state. The supervisor needs no YubiHSM-specific code.

What the host actually talks to

There is no extra network client or server. To macOS, Linux, or Windows the Raspberry Pi is an ordinary USB peripheral. Host class drivers claim interfaces and expose their normal APIs to applications.

USB interface	Host-side path	Typical application
HID / FIDO	OS HID + FIDO stack	Browser, WebAuthn client, libfido2
CCID	USB CCID driver + PC/SC	ykman, GnuPG, OpenSC, pkcs11rs
Trezor vendor interface	Trezor transport library	trezorctl, Trezor Suite
Vendor bulk	libusb or product library	YubiHSM connector/client

Why FIDO can work while CCID is exclusive

HID does not inherently make a device shareable. FIDO and CCID remain independently usable because they are separate USB interfaces claimed by separate host-driver stacks. An application's exclusive PC/SC transaction on the CCID interface does not claim the HID interface.

The same distinction applies inside the worker: each interface has different framing and endpoint semantics even when both ultimately reach applets in one virtual device. HID is report-oriented and host-pollled; CCID and vendor protocols use their declared bulk or interrupt endpoints.

The host never knows about ConfigFS, FunctionFS, SCM_RIGHTS, or worker processes. Those are Linux implementation details behind the USB identity it enumerates.

Pi 4 and Pi 5 use the same model

The deployment is tested on both 64-bit Ubuntu and 64-bit Raspberry Pi OS. On either system, the normal board-specific setup is to enable DWC2 in peripheral mode; the supervisor then uses the resulting UDC through ConfigFS and FunctionFS.

USB gadget mode depends on a USB Device Controller, not on I2C target/peripheral support. Raspberry Pi 4 and Pi 5 both use Linux DWC2 device mode on the gadget-capable USB-C connection. The Pi 5's ordinary RP1 USB host ports are not extra gadget controllers.

```
/sys/class/udc/  
fe980000.usb # typical Pi 4 name  
1000480000.usb # typical Pi 5 name  
  
supervisor: sort names -> choose first  
profile: udc = "EXACT_NAME"
```

When can more than one UDC exist?

- A test system loads dummy_hcd and exposes one or more software controllers.
- A virtualized or emulated platform supplies several device controllers.
- Custom carrier hardware exposes an additional physical device-mode controller.
- A non-Pi system genuinely contains multiple gadget-capable controllers.

One selected UDC exposes one profile at a time. Virtual YubiKey and Virtual Trezor can both be installed, but two simultaneously enumerated identities require two independent controllers or two Pis.

Power and the host port

A Pi 5 can draw substantial power compared with a small USB token. Gadget-mode software does not protect against an inadequate cable, supply, or host-port power budget. A separately powered Pi may still use the data connection, but power topology must avoid unintended back-power paths and follow the board's electrical guidance.

UDC names are discovered from sysfs because they are kernel/hardware identifiers, not stable product names to hard-code in a worker.

The installed contract

Artifact	Owner	Current contract
Profile	Device project; root-installed	Schema 1; worker launch and approved named hardware resources
Supervisor	Privileged service	Validate, construct, open, transfer, bind, monitor, unbind, clean
Worker protocol	Matched deployment set	UGSP version 1; 20-byte headers, typed bodies, generation and request IDs
Worker	Unprivileged device project	Protocol logic, secrets, state, UI, and direct endpoint I/O

Capability boundary

The worker receives open resources, not permission to explore privileged namespaces. FunctionFS remains root-owned, and raw GPIO-chip resources are rejected. The control socket is fixed at FD 3; every other handle arrives through SCM_RIGHTS. The supervisor closes its transferred copies after handoff, while worker process death closes the final copies automatically. The cleared environment contains only the persistent and runtime directory paths, while configured worker options remain command arguments.

This is useful privilege separation, not hardware isolation. A software authenticator or wallet on a general-purpose Pi remains vulnerable to compromise of its worker account, kernel, storage, memory, supply chain, and physical environment.

Operational checks

- Validate the profile before installation and keep the installed copy root-owned.
- Confirm endpoint count, order, direction, and host-visible interface order.
- Observe Configure, UsbEndpoints, Serving, UDC bind, USB activation and teardown logs.
- Kill the worker and verify unbind-first cleanup followed by a fresh process.
- Stop the service and verify final UDC, ConfigFS, mount, display, and process cleanup.

The design is deliberately literal: a virtual firmware implementation reads and writes real kernel endpoint file descriptors. The supervisor supplies those capabilities and controls when the host can see them.

Detailed, browser-readable documentation: [docs/usb-gadget-architecture.md](https://docs.usb-gadget-architecture.md)